FGV EMAp

João Pedro Jerônimo

Engenharia de Software

Revisão para A2

Rio de Janeiro

2026

Conteúdo

1 SOLID - L e I 3
1.1 L - Princípio da Substituição de Liskov 4
1.2 I - Princípio da Segregação de Interfaces 5
2 Chain of Responsibility 7
3 Strategy Pattern 9
4 Command Pattern 11
5 State Pattern 13

SOLID - L e I

1.1 L - Princípio da Substituição de Liskov

O Princípio da Substituição de Liskov (Liskov Substitution Principle - LSP) é um dos cinco princípios do SOLID. O LSP afirma que objetos de uma classe derivada devem poder substituir objetos da classe base **sem alterar a corretude do programa**. Em outras palavras, se uma classe S é uma subclasse de T, então objetos de tipo T podem ser substituídos por objetos de tipo S sem afetar o comportamento esperado do programa.

Imagine que uma empresa criou, internamente, um programa para atender a **todos** os seus usuários. Obviamente, ela vai utilizar da classe base `Usuario` para essa representação. No entanto, em uma empresa madura, com toda certeza vai existir algum tipo de **especialização** de usuários para atender às necessidades da empresa. Para continuar o exemplo, vamos supor a seguinte estrutura:

users.py — python

```python
class Usuario(ABC):
  @abstractmethod
  def gerar_relatorio(self):
    ...

class UsuarioComercial(Usuario):
  def gerar_relatorio(self):
    print("Gerando relatório comercial...")

class UsuarioFinanceiro(Usuario):
  def gerar_relatorio(self):
    raise NotImplementedError("Usuário financeiro não pode gerar relatórios.")
```

Essa estrutura de usuários por si só já é ruim, mas não vamos focar nisso agora. Vamos então montar uma classe para o programa dos nossos usuários!

programa.py — python

```python
class Programa:
  def __init__(self, usuario: Usuario):
    self.usuario = usuario

  def gerar_relatorio(self):
    if isinstance(self.usuario, UsuarioComercial):
      self.usuario.gerar_relatorio()
```

Como eu disse anteriormente, o programa foi feito para atender a todos os usuários, mas ele só leva em conta o usuário comercial. Em primeira vista, podemos até pensar: “Ah, mas o usuário financeiro não precisa gerar relatórios, então tudo bem”. No entanto, se criarmos uma nova classe de usuário, como `UsuarioMarketing` que precisa gerar relatórios, o programa vai quebrar, pois ele não vai saber lidar com esse novo tipo de usuário. Justamente aí que entra a violação do LSP, pois o usuário de marketing pode gerar relatórios, ele é uma subclasse de `Usuario`, então ele poderia e deveria entrar no programa sem o quebrar, mas o programa não foi feito para lidar com ele, então ele quebra. O LSP é violado.

Para consertar a violação do LSP **parcialmente**, vamos fazer uma simples alteração no programa para que ele aceite qualquer tipo de usuário:

programa.py

```python
class Programa:
  def __init__(self, usuario: Usuario):
    self.usuario = usuario

  def gerar_relatorio(self):
    self.usuario.gerar_relatorio()
```

Agora nosso programa está idealmente correto, mas o LSP **ainda é violado**. Mas por quê? Perceba que no nosso arquivo `users.py`, a classe `UsuarioFinanceiro` **não implementa o método** `gerar_relatorio()`, então se passarmos um objeto dessa classe para o programa, ele vai quebrar. O LSP é violado porque a subclasse `UsuarioFinanceiro` não pode substituir a superclasse `Usuario` sem quebrar o programa. No entanto, o LSP está sendo violado em conjunto com outro pricípio do SOLID. Antes de implementarmos a devida correção, vamos falar sobre o outro princípio que está sendo violado.

1.2 I - Princípio da Segregação de Interfaces

O Princípio da Segregação de Interfaces (Interface Segregation Principle - ISP) é outro princípio do SOLID que afirma que **nenhum cliente deve ser forçado a depender de métodos que não utiliza**. Em outras palavras, é melhor ter várias interfaces específicas do que uma interface única e genérica. O ISP incentiva a criação de interfaces menores e mais focadas, permitindo que os clientes implementem apenas os métodos que realmente precisam, evitando a implementação de métodos desnecessários.

Vamos rever o nosso arquivo `users.py` para entender como o ISP está sendo violado:

users.py

```python
class Usuario(ABC):
  @abstractmethod
  def gerar_relatorio(self):
    ...

class UsuarioComercial(Usuario):
  def gerar_relatorio(self):
    print("Gerando relatório comercial...")

class UsuarioFinanceiro(Usuario):
  def gerar_relatorio(self):
    raise NotImplementedError("Usuário financeiro não pode gerar relatórios.")
```

Nós haviamos dito anteriormente que **todos** os usuários deveriam ter acesso ao nosso programa! Para isso, precisamos que todos eles possam gerar relatório, entretanto, nosso usuário financeiro não pode gerar relatórios. **Vamos supor** que em nosso cenário, isso foi erro de modelagem do programa, e usuários financeiros **realmente não podem gerar relatórios**. O ISP está sendo violado pois, mesmo que o usuário financeiro não precise gerar relatórios, ele é forçado a implementar o método `gerar_relatorio()`, mesmo que não faça sentido para ele. Para corrigir

isso, vamos criar uma nova interface chamada `UsuarioComRelatorio`, e o método de geração de relatórios vai ser movido para essa nova interface. Assim, apenas os usuários que realmente precisam gerar relatórios vão implementar esse método.

users.py — python

```python
class Usuario(ABC):
  pass

class UsuarioComRelatorio(Usuario):
  @abstractmethod
  def gerar_relatorio(self):
    ...

class UsuarioComercial(UsuarioComRelatorio):
  def gerar_relatorio(self):
    print("Gerando relatório comercial...")

class UsuarioFinanceiro(Usuario):
  pass
```

Então mudamos a abordagem do nosso programa, em vez de ele ficar disponível para **qualquer usuário**, ele vai ficar disponível apenas para **usuários que podem gerar relatórios**.

programa.py — python

```python
class Programa:
  def __init__(self, usuario: UsuarioComRelatorio):
    self.usuario = usuario

  def gerar_relatorio(self):
    self.usuario.gerar_relatorio()
```

Dessa forma, o LSP e o ISP foram corrigidos, e nosso programa agora está idealmente correto. O LSP foi corrigido pois agora qualquer subclasse de `UsuarioComRelatorio` pode substituir a super-classe sem quebrar o programa, e o ISP foi corrigido pois agora apenas os usuários que realmente precisam gerar relatórios implementam o método `gerar_relatorio()`, evitando a implementação de métodos desnecessários.

Chain of Responsibility

Strategy Pattern

Command Pattern

State Pattern